

Preliminary Progress Review (PPR) Report of Project CAPSLock (25-26J-043)

Project ID: 25-26J-043

**Project Title: CAPSLock – Security Gateway for Kubernetes,
DevOps and Pipelines**

Student Details

Names:	KULATHUNGA K K M D RAIGAMBANDARAGE N K GURUGE S A L PANDUKABHAYA H D T
Student IDs:	IT22347626 IT22338716 IT22345028 IT22353634
Supervisor:	Mr. Amila Senarathna
Date of Submission:	January 11, 2026

1. Table of Contents and Index

1. Table of Contents and Index	2
2. Introduction.....	3
2.1. Purpose.....	3
2.2. Definitions, Acronyms, and Abbreviations	3
2.3. Overview.....	4
3. Literature review	4
3.1. Background and Current State	4
3.2. Identification and significance of the problem	5
4. Research Question	5
5. Research Objectives.....	6
6. Research methodology.....	7
6.1. Conceptual Framework	7
6.2. Implementation Methodology.....	7
7. Sources for test data and analysis	8
8. Anticipated benefits	9
8.1. Benefits to Users	9
8.2. Contribution to the Body of Knowledge.....	9
9. Scope and specified deliverables / expected research outcome.....	10
9.1. In Scope	10
9.2. Out of Scope	11
9.3. Expected Research Outcomes.....	11
10. Research constraints (if any).....	12
11. Project plan or schedule	13
12. References	15
13. Appendices.....	16
Appendix A: Component Progress Summary.....	16
Appendix B: Integration Contract Summary	17

2. Introduction

2.1. Purpose

This Preliminary Progress Review report documents the current state of development for the CAPSLock project at the 50% completion milestone. The report serves to communicate progress across all four system components, identify integration achievements and challenges, assess adherence to the project timeline, and provide transparency regarding technical decisions and deviations from the original plan. The intended audience comprises the project supervisor, academic evaluators, and stakeholders within the Information Assurance and Security research group at SLIIT who require comprehensive understanding of the project's trajectory toward completion.

This document substantiates that substantial foundational work has been accomplished across the Multi-Environment Deployment System, Environment-Aware Policy Engine, Service Discovery and Load Balancing component, and Kubernetes ICAP Operator. Furthermore, it delineates the remaining work required to achieve full system integration, comprehensive testing, performance optimization, and documentation completion necessary for successful project delivery.

2.2. Definitions, Acronyms, and Abbreviations

<i>Acronym</i>	<i>Definition</i>
<i>CAPSLock</i>	<i>Container-Aware Policy Security Lock</i>
<i>ICAP</i>	<i>Internet Content Adaptation Protocol (RFC 3507)</i>
<i>MEDS</i>	<i>Multi-Environment Deployment System</i>
<i>EAPE</i>	<i>Environment-Aware Policy Engine</i>
<i>SDLB</i>	<i>Service Discovery and Load Balancing</i>
<i>CRD</i>	<i>Custom Resource Definition</i>
<i>OPA</i>	<i>Open Policy Agent</i>
<i>API</i>	<i>Application Programming Interface</i>
<i>WSL2</i>	<i>Windows Subsystem for Linux 2</i>
<i>NIST</i>	<i>National Institute of Standards and Technology</i>
<i>PDPA</i>	<i>Personal Data Protection Act</i>
<i>PCI-DSS</i>	<i>Payment Card Industry Data Security Standard</i>
<i>SOC2</i>	<i>Service Organization Control 2</i>

2.3. Overview

This PPR report is organized into ten principal sections that collectively provide comprehensive documentation of the CAPSLock project's progress. Following this introduction, Section 2 presents an extensive literature review establishing the research context and identifying gaps in existing container security solutions. Section 3 articulates the research problem's significance and novelty. Section 4 delineates the research questions and objectives guiding the investigation. Section 5 describes the research methodology employed across all four components.

Section 6 examines the anticipated benefits to users and contributions to the body of knowledge in container security. Section 7 defines the project scope and specified deliverables. Section 8 addresses research constraints encountered during development. Section 9 presents the detailed project schedule with milestone achievements. Section 10 provides comprehensive references to academic literature, technical documentation, and industry standards that inform the research.

3. Literature review

3.1. Background and Current State

Elgendi et al. (2019) provide comprehensive analysis of container security challenges, identifying architectural vulnerabilities inherent in shared kernel architectures that differentiate containers from traditional virtual machines. The authors document prevalent attack vectors including container escape vulnerabilities, image tampering, and insecure network configurations. Their work establishes that while containers offer operational efficiency advantages, they introduce unique security considerations requiring specialized protection mechanisms.

Wong et al. (2024) conduct systematic survey of container security exploits, attacks, and defense mechanisms, revealing that container security research has expanded significantly since 2015 yet remains fragmented across multiple domains including image security, runtime protection, network isolation, and orchestration security. The authors identify that integration challenges between security tools and container orchestration platforms constitute a persistent gap in current solutions.

Rahman et al. (2022) examine security misconfigurations in open-source Kubernetes manifests through empirical analysis, discovering prevalent instances of privilege escalation vulnerabilities, excessive permissions, and inadequate network policies. Their findings demonstrate that policy enforcement complexity in multi-environment Kubernetes deployments contributes substantially to security misconfigurations. The research underscores the necessity for automated, environment-aware policy management systems.

Senjab et al. (2023) survey Kubernetes scheduling algorithms, revealing that scheduling decisions significantly impact both performance and security posture. The authors document challenges in uniform policy enforcement across heterogeneous cluster deployments, particularly when workloads span multiple environments with varying security requirements. This research motivates investigation into environment-aware security policy adaptation mechanisms.

3.2. Identification and significance of the problem

The CAPSLock project addresses three critical research gaps identified through comprehensive literature analysis. First, existing ICAP implementations designed for traditional HTTP proxy architectures prove incompatible with ephemeral, dynamically scheduled container lifecycles characteristic of Kubernetes environments. RFC 3507 specification assumes persistent connections and static service endpoints; these assumptions fail in containerized deployments where pods are created and terminated continuously based on workload demands.

Second, current container security tools operate in isolation, lacking standardized integration frameworks that would enable unified security orchestration. Organizations deploying container platforms must integrate disparate security solutions through proprietary APIs, resulting in vendor lock-in and operational complexity. The absence of standards-based integration mechanisms impedes comprehensive security coverage across the container lifecycle.

Third, multi-environment policy management complexity represents a significant operational challenge. Empirical research demonstrates that security policies applied uniformly across development, staging, and production environments either impose excessive restrictions that impede development velocity or provide insufficient protection in production deployments. The literature lacks systematic approaches to automated, environment-aware security policy adaptation that balances security requirements with operational efficiency across deployment contexts.

The scientific and technical challenge resides in designing a comprehensive system that integrates ICAP security services natively within Kubernetes orchestration while providing intelligent, automated policy adaptation across multi-environment deployments. This integration requires solving problems in distributed systems coordination, policy conflict resolution, performance optimization under containerized workload patterns, and transparent traffic interception without disrupting application functionality.

4. Research Question

How can Internet Content Adaptation Protocol (ICAP) security services be integrated as a native component of Kubernetes container orchestration to provide automated, environment-aware policy enforcement across multi-environment deployments while maintaining operational efficiency and scalability?

5. Research Objectives

5.1. Main Objective:

To design, implement, and evaluate a comprehensive system that integrates ICAP security services as a core component of container orchestration, providing automated health monitoring, intelligent load balancing, environment-aware policy management, and lifecycle orchestration across multi-environment Kubernetes deployments.

5.2. Specific Objectives:

- a) Design and implement a Kubernetes-native ICAP operator that provisions and manages c-icap and ClamAV pods with automated health monitoring, scaling capabilities, and lifecycle management within container orchestration environments.
- b) Develop an environment-aware policy engine that automatically detects deployment contexts through Kubernetes metadata analysis and enforces appropriate security policies through integration with OPA Gatekeeper and Kyverno policy frameworks.
- c) Implement service discovery and load balancing mechanisms that integrate with Kubernetes DNS and Istio service mesh to provide transparent traffic interception and distributed ICAP service management with health-aware routing algorithms.
- d) Create a multi-environment deployment system that manages unified policy deployment and orchestration across development, staging, and production Kubernetes environments with automated promotion pipelines and rollback capabilities.

6. Research methodology

6.1. Conceptual Framework

The CAPSLock conceptual framework comprises four integrated components that collectively address the research objectives. The framework establishes formal contracts between components defining interfaces, data exchange protocols, and behavioral expectations. Component integration follows a layered architecture wherein the Multi-Environment Deployment System orchestrates high-level operations, the Environment-Aware Policy Engine provides intelligent decision-making, the Service Discovery and Load Balancing component manages traffic distribution, and the Kubernetes ICAP Operator handles service provisioning.

Key variables measured across the system include environment detection accuracy, policy selection correctness, conflict resolution effectiveness, service discovery latency, load balancing fairness, pod provisioning time, and end-to-end security enforcement latency. These variables enable quantitative assessment of system performance against established benchmarks.

6.2. Implementation Methodology

Implementation follows an iterative development methodology with two-week sprint cycles. Each sprint encompasses requirements analysis, design refinement, implementation, unit testing, and integration testing activities. The development environment utilizes K3s lightweight Kubernetes distribution running on Windows WSL2 for local development, with multi-node cloud clusters employed for integration and performance testing.

Component 1 (MEDS) implementation employs Go programming language with Kubernetes client-go libraries for multi-cluster communication. The component utilizes GitOps principles with Git repositories serving as the source of truth for policy configurations. Component 2 (EAPE) leverages the controller-runtime framework for Kubernetes controller development, implementing policy selection algorithms with weighted scoring mechanisms and conflict resolution logic based on precedence rules.

Component 3 (SDLB) integrates with Istio service mesh through VirtualService and DestinationRule custom resources, implementing health-aware load balancing algorithms that consider CPU utilization, latency metrics, and error rates. Telemetry collection utilizes Prometheus with PromQL queries for continuous performance monitoring. Component 4 (Kubernetes ICAP Operator) employs Operator SDK for custom resource definition and controller scaffolding, managing c-icap and ClamAV pod lifecycles through Deployment resources.

7. Sources for test data and analysis

Test data comprises synthetic Kubernetes workloads representative of typical microservices architectures, including web applications, API services, and data processing pipelines. Container images with known vulnerabilities from public registries enable validation of ICAP scanning effectiveness. Performance benchmarking employs load generation tools to simulate realistic traffic patterns across multiple environment types.

Data collection procedures utilize Prometheus for time-series metrics, Grafana for visualization, and custom logging frameworks for audit trail generation. Key performance indicators include environment detection latency (target < 100ms), policy selection throughput (target > 500 evaluations/second), service discovery latency (target < 50ms), and pod provisioning time (target < 30 seconds). Statistical analysis employs percentile distributions (P50, P95, P99) to characterize system behavior under varying load conditions.

System validation encompasses functional correctness verification, performance benchmarking, security effectiveness assessment, and comparative evaluation against existing solutions. Functional correctness testing validates that environment detection correctly identifies deployment contexts, policy selection applies appropriate security configurations, conflict resolution produces consistent outcomes, and ICAP scanning detects known vulnerabilities.

Performance benchmarking measures system behavior under varying workload intensities, cluster sizes, and failure conditions. Security effectiveness assessment employs penetration testing methodologies to validate that the system successfully intercepts and scans container traffic without introducing bypass vulnerabilities. Comparative evaluation positions CAPSLock against commercial container security platforms to demonstrate novel capabilities and identify areas for further optimization.

8. Anticipated benefits

8.1. Benefits to Users

Organizations deploying containerized applications on Kubernetes will benefit from standardized integration of ICAP security services without requiring proprietary security gateway appliances. The automated, environment-aware policy enforcement reduces operational overhead associated with manual security configuration management across development, staging, and production environments. Development teams gain flexibility to iterate rapidly in development environments while production deployments maintain stringent security controls automatically.

The transparent traffic interception approach via service mesh integration eliminates the necessity for application code modifications to incorporate security scanning. Applications maintain security enforcement mechanisms, enabling security teams to update and enhance protection capabilities independently of application development cycles. The unified management interface provides centralized visibility into security policy enforcement across multi-cluster, multi-environment deployments.

Compliance-aware routing capabilities ensure that workloads handling sensitive data are automatically directed to ICAP instances meeting requisite compliance standards such as PCI-DSS or SOC2, reducing compliance burden and audit complexity. Automated health monitoring and failover mechanisms enhance system reliability, ensuring continuous security coverage even during infrastructure failures or maintenance operations.

8.2. Contribution to the Body of Knowledge

This research makes several significant contributions to the understanding of container security. First, it establishes a novel integration architecture bridging RFC 3507 ICAP protocol with Kubernetes-native orchestration patterns. This contribution addresses a gap identified in comprehensive literature analysis revealing the absence of scholarly work on ICAP orchestration within container platforms.

Second, the environment-aware policy engine introduces a systematic approach to automated security policy adaptation based on deployment context. The weighted scoring algorithm and conflict resolution mechanisms provide reusable patterns applicable beyond the immediate CAPSLock implementation to broader domains of multi-environment configuration management. The research establishes that intelligent policy adaptation can effectively balance security requirements with operational efficiency across environment boundaries.

Third, the container-aware load balancing algorithms optimized for ICAP service distribution patterns contribute novel insights into performance optimization in containerized security scanning architectures. The integration of health metrics, compliance requirements, and traffic patterns into routing decisions represents advancement over traditional round-robin or least-connections load balancing approaches.

Fourth, the comprehensive system architecture demonstrating end-to-end integration of security services within container orchestration establishes a reference architecture that other researchers and practitioners can adapt to integrate different security tools or content adaptation services with Kubernetes. The formal integration contracts, API specifications, and operator patterns developed in this research provide reusable building blocks for future container security research.

9. Scope and specified deliverables / expected research outcome

9.1. In Scope

The CAPSLock system will provide the following capabilities:

- Automated deployment and lifecycle management of c-icap and ClamAV security scanning services within Kubernetes clusters through custom operator implementation
- Environment detection capabilities analyzing Kubernetes namespace labels and annotations to identify development, staging, and production contexts with confidence scoring
- Automated policy selection applying appropriate security configurations based on detected environment, compliance requirements, and risk tolerance specifications
- Policy conflict detection and resolution mechanisms employing precedence-based rules and configurable resolution strategies
- Integration with OPA Gatekeeper and Kyverno policy engines through automated generation of ConstraintTemplate and ClusterPolicy resources
- Transparent traffic interception and routing to ICAP services via Istio service mesh integration without requiring application code modifications
- Health-aware load balancing across multiple ICAP service instances considering CPU utilization, latency metrics, error rates, and compliance requirements
- Multi-cluster policy deployment and orchestration capabilities enabling unified management across development, staging, and production Kubernetes clusters
- Policy promotion pipelines with automated validation and rollback mechanisms
- RESTful API for programmatic interaction with policy management, environment detection, and system monitoring capabilities
- Web-based dashboard providing visualization of policy enforcement status, service health metrics, and traffic routing patterns
- Comprehensive audit logging recording all policy decisions, enforcement actions, and system state changes
- Integration with Prometheus for metrics collection and Grafana for monitoring visualization

9.2. Out of Scope

The following capabilities are explicitly excluded from the current project scope:

Development of custom ICAP scanning engines; the system utilizes existing c-icap and ClamAV implementations

Support for non-Kubernetes container orchestration platforms such as Docker Swarm or Apache Mesos

Integration with service meshes other than Istio; alternative service mesh support would require additional development

Automated machine learning-based threat detection; the system focuses on integration and orchestration rather than detection algorithm development

Production-grade high availability configurations spanning multiple geographic regions; the focus remains on functional correctness and single-region deployment scenarios

Comprehensive cost optimization algorithms for cloud deployment; resource optimization focuses on functional efficiency rather than cost minimization

9.3. Expected Research Outcomes

Functional prototype system demonstrating end-to-end integration of all four components

Comprehensive technical documentation including architecture diagrams, API specifications, deployment procedures, and troubleshooting guides

Performance benchmark results quantifying system behavior under various load conditions

Security effectiveness evaluation demonstrating successful detection and blocking of containerized threats

Comparative analysis positioning CAPSLock against existing container security solutions

Research paper documenting the integration architecture, algorithms developed, and empirical evaluation results suitable for submission to academic conferences or journals

Open-source code repository enabling reproducibility and facilitating future research extensions

10. Research constraints (if any)

Several constraints influence the project scope, methodology, and anticipated outcomes. First, the computational resource availability limits the scale of testing environments. While cloud resources are available for development and testing, budget constraints preclude large-scale, multi-region deployment testing that would fully validate the system's scalability characteristics. Performance benchmarks must be conducted on moderately sized clusters rather than enterprise-scale infrastructures.

Secondly, integration testing complexity increases exponentially with the number of components and their interdependencies. Comprehensive integration testing across all possible failure scenarios, edge cases, and configuration permutations would require testing resources beyond project constraints. Testing strategy must prioritize critical paths and common scenarios while acknowledging that exhaustive testing remains infeasible.

Next, dependency on external technologies including Kubernetes, Istio, OPA Gatekeeper, and Kyverno introduces version compatibility considerations. The project must accommodate API changes, deprecated features, and evolving best practices in rapidly developing cloud-native ecosystems. This constraint requires balancing current best practices with long-term maintainability considerations.

Finally, evaluation methodology limitations stem from the difficulty of obtaining realistic production workload data for testing purposes. Synthetic workload generation provides controlled testing scenarios but may not fully represent the diversity and complexity of real-world containerized applications. Security effectiveness evaluation must rely on known vulnerability datasets rather than comprehensive threat modeling against emerging attack vectors.

11. Project plan or schedule

11.1. Completed Phases

Phase 1: Project Planning and Setup

- Comprehensive literature review and research gap analysis
- Architecture design and component boundary definition
- Integration contract specification between components
- Development environment setup including K3s installation, tool configuration, and repository initialization
- Kubernetes fundamentals study including pods, services, namespaces, and custom resource definitions

Phase 2: Core Component Development

Component 1 (MEDS):

- Multi-cluster deployment architecture implementation
- Environment promotion pipeline framework development
- Policy migration logic with GitOps integration
- Unified management interface prototype creation

Component 2 (EAPE):

- Environment detection system using Kubernetes API
- Policy template schema design and baseline template creation
- Policy manager with template loading and validation
- Policy selection algorithm with weighted scoring
- Conflict detection and resolution mechanisms
- OPA Gatekeeper and Kyverno integration with resource generation
- RESTful API implementation and dashboard prototype

Component 3 (SDLB):

- Automatic service registration with Kubernetes DNS
- Core load balancing decision engine development
- Health monitoring framework with configurable checks
- Prometheus telemetry collection integration
- Istio VirtualService and DestinationRule generation
- Compliance-aware routing logic implementation

Component 4 (Kubernetes ICAP Operator):

- IcapService Custom Resource Definition design
- Operator controller logic using controller-runtime
- Automatic c-icap and ClamAV pod deployment
- Health check probe configuration
- Istio sidecar injection and traffic routing
- Basic auto-scaling based on resource metrics

Phase 3: Initial Integration and Testing

- Mock component development for isolated testing
- Component-to-component integration testing
- Initial performance benchmarking
- Bug identification and resolution
- 50% progress review report preparation and submission
- 8.2 Remaining Phases (Weeks 11-20)

Phase 4: Comprehensive Integration

- End-to-end integration testing with all components
- Integration contract refinement based on testing insights
- Error propagation and handling mechanisms
- Synchronization and consistency verification
- Failure scenario testing and resilience validation

Phase 5: Performance Optimization and Load Testing

- Comprehensive load testing under realistic workload scenarios
- Performance bottleneck identification and optimization
- Resource utilization analysis and tuning
- Scalability testing across varying cluster sizes
- Comparative evaluation against existing container security solutions

Phase 6: Security Evaluation

- Vulnerability detection effectiveness assessment
- Penetration testing to identify bypass vulnerabilities
- Compliance validation for PDPA, PCI-DSS, and SOC2 requirements
- Security audit log verification

Phase 7: Documentation and Deployment Guides

- Comprehensive architecture documentation with diagrams
- API specification completion with OpenAPI documentation
- Deployment procedures and configuration guides
- User documentation including quickstart guides and tutorials
- Troubleshooting guides and operational procedures
- Code documentation and inline comment refinement

Phase 8: Final Demonstration and Report

- Demonstration script preparation showcasing all system capabilities
- Presentation materials development for final defense
- Final project report compilation integrating all documentation
- Research paper preparation for potential conference submission
- Code repository finalization and open-source release preparation
- Final submission and project handover

12. References

1. Elgendi, I., Mustapha, A., Nasir, Q., & Marka, A. (2019). Container Security: Issues, Challenges, and the Road Ahead. *IEEE Access*, 7, 52976-52996.
<https://doi.org/10.1109/ACCESS.2019.2912726>
2. Senjab, K., Abbas, S., Ahmed, N., & Khan, A. U. R. (2023). A Survey of Kubernetes Scheduling Algorithms. *Journal of Cloud Computing*, 12(1), 87.
<https://doi.org/10.1186/s13677-023-00471-1>
3. Wong, A. Y., Hartner, M., Johal, G., & Liu, Y. (2024). A Container Security Survey: Exploits, Attacks, and Defenses. *ACM Computing Surveys*, 57(4), 1-36.
<https://doi.org/10.1145/3679015>
4. Sultan, S., Ahmad, I., & Dimitriou, T. (2019). Container Security: Issues, Challenges, and the Road Ahead. *IEEE Access*, 7, 52976-52996.
<https://doi.org/10.1109/ACCESS.2019.2912726>
5. Rahman, A., Parnin, C., & Williams, L. (2022). Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study. *ACM Transactions on Software Engineering and Methodology*, 31(4), 1-50. <https://doi.org/10.1145/3503463>
6. Rahman, M. A., Hassanpour, M., Abdelzaher, T., & Gupta, I. (2022). Kubernetes Scheduling: Taxonomy, Ongoing Issues and Challenges. *ACM Computing Surveys*, 55(4), 1-37. <https://doi.org/10.1145/3495245>
7. Kubernetes Documentation. (2025). Custom Resources. Retrieved from <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>
8. Istio Documentation. (2025). Traffic Management. Retrieved from <https://istio.io/latest/docs/concepts/traffic-management/>
9. Open Policy Agent Documentation. (2025). Gatekeeper Policy Controller. Retrieved from <https://open-policy-agent.github.io/gatekeeper/>
10. Kyverno Documentation. (2025). Kubernetes Native Policy Management. Retrieved from <https://kyverno.io/docs/>
11. Cooper, M., Moore, J., Cooper, D., & Elkins, S. (2003). RFC 3507: Internet Content Adaptation Protocol (ICAP). Internet Engineering Task Force. Retrieved from <https://www.rfc-editor.org/rfc/rfc3507>
12. Cloud Native Computing Foundation. (2024). Kubernetes Operator Pattern. Retrieved from <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>
13. Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5), 50-57.
<https://doi.org/10.1145/2890784>
14. National Institute of Standards and Technology. (2017). Application Container Security Guide (NIST Special Publication 800-190). U.S. Department of Commerce. Retrieved from <https://csrc.nist.gov/publications/detail/sp/800-190/final>

13. Appendices

Appendix A: Component Progress Summary

Component	Team Member	Key Achievements
MEDS (Component 1)	IT22353634	Multi-cluster architecture, promotion pipelines, unified management interface
EAPE (Component 2)	IT22338716	Environment detection, policy engine, OPA/Kyverno integration, REST API
SDLB (Component 3)	IT22345028	Service discovery, load balancing, health monitoring, Istio integration
ICAP Operator (Component 4)	IT22347626	CRD design, operator controller, pod lifecycle management, auto-scaling

Appendix B: Integration Contract Summary

Formal integration contracts have been established between components to ensure consistent communication protocols and data exchange formats. The following contracts represent critical integration points:

Contract 1: MEDS to EAPE (Namespace Labeling)

Required namespace labels: environment={dev|staging|prod}, security-level={low|medium|high}, compliance={comma-separated standards}. Optional annotations for additional metadata. EAPE consumes these labels for environment detection and policy selection.

Contract 2: EAPE to ICAP Operator (Policy Enforcement)

EAPE generates IcapService custom resource specifications based on selected policies. The ICAP Operator reconciles these specifications by creating or updating c-icap and ClamAV pod deployments. Configuration parameters include scanning mode, file size limits, virus scanning settings, and content filtering rules.

Contract 3: ICAP Operator to SDLB (Service Registration)

The ICAP Operator creates Kubernetes Service resources for provisioned ICAP instances. SDLB automatically discovers these services through Kubernetes API watchers and registers them in its service registry. Health status is communicated through service endpoint conditions.

Contract 4: MEDS to SDLB (Environment Metadata)

MEDS provides environment metadata including compliance requirements and risk tolerance specifications. SDLB utilizes this information for compliance-aware routing decisions, ensuring sensitive workloads are directed to appropriately configured ICAP instances.